SOFTWARE REQUIREMENTS SPECIFICATION

Library Management System

PREPARED BY	**Mim Akter** ID: 251035005
DEPARTMENT	Artificial Intelligence and Data Science (ADS)
COURSE	Software Engineering / Database Systems
DATE	June 2026

Table of Contents

1. Introduction

1.1 Purpose

The purpose of the Library Management System (LMS) is to automate the management of library resources, member records, book borrowing, book returns, fine calculations, and report generation. The system aims to reduce manual paperwork and significantly improve overall operational efficiency within educational or corporate resource centers.

1.2 Scope

The Library Management System is a comprehensive web-based application that enables administrators, librarians, and students to manage library activities seamlessly from anywhere.

The system will natively provide:

- User authentication and role-based authorization
- Book inventory management (cataloging, tracking, and auditing)
- Member database management
- Book borrowing and returning transaction handlers
- Automated fine calculation engines
- Multi-criteria search functionalities
- Reporting, administrative dashboards, and audit-ready analytics

The system will be systematically engineered using React.js for the presentation layer, Node.js and Express.js for the middleware API backend services, and MySQL as the relational database storage framework.

1.3 Intended Audience

This document is structured for review and utilization by the following stakeholders:

- **Students:** For operating guidelines and workflow definitions.
- **Project Supervisors:** For assessment of functional and architectural criteria.
- **Software Developers:** As a system baseline implementation blue-print.
- **Database Designers:** To map entities, definitions, and schemas.
- **Testers:** To design formal system verification test cases.

1.4 Definitions

Term	Meaning
LMS	Library Management System
Admin	System administrator responsible for structural configuration and high-level controls.
Member	Student or authorized system user who holds borrowing privileges.
Librarian	Operational staff person responsible for day-to-day book inventory and queue management.
DBMS	Database Management System
API	Application Programming Interface

2. Overall Description

2.1 Product Perspective

The Library Management System functions as a standalone web application designed to replace obsolete offline bookkeeping frameworks. It organizes operations into a scalable distributed paradigm architecture consisting of:

- **Frontend Interface:** Created in React.js, delivering a single-page reactive application flow.
- **Backend API Layer:** Driven by Node.js & Express.js to process decoupled RESTful instructions securely.
- **Database Layer:** Structured dynamically inside a MySQL Relational Engine.

2.2 Product Functions

The primary high-level functionalities executed by the system encompass:

- Comprehensive Book Inventory Cataloging
- Member Registration and Life-cycle Management
- Librarian Role Allocations
- Real-time Issuance and Check-in Handlers
- Overdue Book Return Tracking and Automated Fine Assessment
- Dynamic Global Catalogs and Indexed Search Pipelines
- Analytical Reporting Engine
- Secure Identity Sign-on System

2.3 User Classes

2.3.1 Administrator

Responsibilities: High-level system maintenance. Controls internal user provisioning, system wide parameters, configures fine rates, audits activities, views aggregate dashboard analytics, and pulls core reports.

2.3.2 Librarian

Responsibilities: Operational catalog operations. Performs manual creation, modification, or removal of titles; verifies asset allocation; issues and receives books; signs off on fine clearances.

2.3.3 Member

Responsibilities: End-user consumption workflows. Searches active collections, reviews historical tracking records, tracks unpaid balances, updates customized user settings, and signs up for notifications.

2.4 Operating Environment

The software suite will operate reliably across the following baseline frameworks:

- **Frontend Stack:** React.js, Tailwind CSS
- **Backend Execution Engine:** Node.js, Express.js Middleware Framework
- **Data Architecture:** MySQL Server
- **Client Interoperability:** Google Chrome, Mozilla Firefox, Microsoft Edge, Apple Safari

3. Functional Requirements

FR-1: User Registration

Description:	Allows new prospective members or system profiles to establish structured login credentials.
Input parameters:	Full Name, Valid Email Address, Secure Password.
Expected Output:	Persisted unique system profile record and successful registration response message.

FR-2: User Login

Description:	Validates user profile authenticity to grant contextual application dashboard workspace permissions.
Input parameters:	Registered Email Address, Verified Password.
Expected Output:	Cryptographic validation token (JWT), UI layout initialization corresponding to assigned role permissions.

FR-3: Book Management

Enables structural maintenance of the centralized repository catalog. The system restricts modification profiles strictly to the Librarian role.

Supported Procedures: Insert New Book Assets, Modify Textual Elements, Purge Items From Inventory, Pull Unified System Indexes.

Book Record Field Structure:

- Title (String index alphanumeric representation)
- Author (Primary and secondary creator categorization)
- ISBN (International Standard Book Number constraint)
- Category (Genre, research domain, or reference labeling)
- Quantity (Integer allocation metrics tracking in-stock limits)

FR-4: Member Management

Grants capabilities to Librarian or Administrator identities to oversee systemic usage permissions.

Supported Actions: Enroll New Members, Patch Outdated Contact Profiles, Terminate Member Accounts, View Member Borrowing Profiles.

FR-5: Search Books Catalog Pipeline

Exposes optimized lookup indexes accessible across all authenticated and unauthenticated browser profiles to isolate relevant assets quickly.

Available Query Parameters: Full / Partial Book Title, Author Name, Distinct ISBN Numeric Identifiers, Genre Categories.

FR-6: Issue Book Allocation

Manages automated checking-out processes for library book allocations. Validates user account metrics prior to completing the database transaction.

Process Flow Sequence:

1. Identify and pull targeted Member account file.
2. Query physical Book Asset availability checklist.
3. Confirm volume balance counter is > 0.
4. Generate transaction log stamping Issue Timestamp and Expected Return Deadline.

Expected Output: Incremental transaction log generation and decrement of available storage quantity.

FR-7: Return Book Processing

Coordinates processing of returned physical assets to clear balances on individual records.

Process Flow Sequence:

1. Locate active open-ended Borrowing Record file.
2. Execute Check-in event.
3. Pass records through automated Fine Computation Check rules engine.
4. Increment physical warehouse asset availability registers.

Expected Output: Terminal status update of historical record; instant adjustment of operational stock levels.

FR-8: Fine Management

The system automatically executes asynchronous background processes to update account liabilities whenever assets bypass expected return dates.

$$Fine = Number\ of\ Overdue\ Days \times Daily\ Fine\ Rate$$

FR-9: Borrowing History

Provides Members with tracking access to view chronological application engagement history. Data visibility is restricted strictly to the logged-in owner's profile.

Displayed Tracking Parameters: Complete List of Borrowed Titles, Target Return Timestamps, Status Flags (Active, Returned, Overdue), Accrued Fine Liabilities.

FR-10: Reporting System

Compiles structured business intelligence matrix outputs regarding application use patterns.

Output Types: Daily Activity Summary Logs, Weekly Operations Snapshots, Monthly Corporate Analytical Data Sheets, Outstanding Fine Liability Reports, Resource Utilization Frequency Records.

4. External Interface Requirements

4.1 User Interface

The application interface will follow modern human-interface visual clean guidelines using minimal styling layout paradigms:

- **Login Interface Page:** Structured form containing user identity text input entry fields, masking security string fields for Password entries, and explicit contextual access action triggers.
- **Dashboard Environment:** Provides real-time metrics tracking displays via informational numerical KPI indicators, interactive fast access operation tools, and customized reporting view frames.
- **Book Inventory Workspace:** Centralized interface console supporting responsive dialog popups to input resource text strings, item editing interfaces, and unified search entry bars.
- **Borrowing Control Console:** Specialized task pane allowing transaction dispatching, barcode input fields for manual operations, and list view filters showing open circulation pipelines.

4.2 Hardware Requirements

To establish minimal acceptable operating configurations for system hosting and end-point clients, the following guidelines are established:

4.2.1 Minimum Specifications

- Processor Architecture: Intel Core i3 Dual-Core clock framework or equivalent architecture.
- System RAM: 4 Gigabytes (GB) structural memory space.
- Storage Availability: 500 Megabytes (MB) of dedicated block-level system workspace.

4.2.2 Recommended Specifications

- Processor Architecture: Intel Core i5 Quad-Core processor matrix or higher computing equivalents.
- System RAM: 8 Gigabytes (GB) volatile random-access space.
- Storage Availability: Solid State Disk (SSD) configurations optimized for high IOPS performance.

4.3 Software Requirements

The platform construction framework utilizes the following software utilities for maintenance and lifecycle development:

- **IDE Infrastructure:** Visual Studio Code
- **Database Modeling Client:** MySQL Workbench Server Admin Tools
- **Source Control Middleware:** Distributed Git Engine mirrored to remote GitHub Repositories
- **API Prototyping Suites:** Postman API testing automation suites

5. Non-Functional Requirements

Category	Requirement Metric Specification
Performance	Search operations must process indexes and output data structures within 2 seconds. The system must natively scale to handle inventories containing at least 1,000 distinct items without query degradation.
Security	Passwords must undergo irreversible cryptographic encryption before storage. Client sessions must be validated using signed JSON Web Tokens (JWT) adhering to strict role-based access rules.
Reliability	Ensures ACID compliance across relational records to maintain database consistency. Automates scheduled system data snapshot backups every 24 hours.
Scalability	The system structure must remain decoupled to support future traffic growth and additional functional components smoothly.
Availability	Guarantees a 99% operational uptime standard, excluding pre-announced architectural maintenance windows.
Maintainability	The codebase must follow clean design principles, modular folder separations, and clear internal inline comments.

6. Use Cases

Use Case 1: Secure System Authentication Login

Primary Actor: Registered System User (Member, Librarian, or Admin)

Precondition: The client profile record must exist inside the MySQL user registry table.

Step-by-step Process:

1. The user opens the standard web application login page.
2. The user enters their registered Email Address string.
3. The user inputs their security Password characters.
4. The user triggers the formal authentication action interface.
5. The application system validates the profile data and displays the corresponding role dashboard.

Postcondition: Authorization context tokens are safely stored in browser storage, granting access to application paths.

Use Case 2: Asset Borrowing Processing

Primary Actor: Authenticated Librarian

Step-by-step Process:

1. The Librarian selects the targeted Member account within the system interface.
2. The Librarian searches the system catalog to isolate the requested Book volume record.
3. The application engine verifies the availability status metrics of the chosen item.
4. The Librarian triggers the operational Issue Book confirmation instruction.
5. The application stores the transactional relationship parameters securely in the database.

Postcondition: The borrowing transaction record is created successfully, and the system decreases the available inventory count by one.

Use Case 3: Asset Check-in and Return Clearance

Primary Actor: Authenticated Librarian

Step-by-step Process:

1. The Librarian opens the operational returns interface and searches for the matching open Borrow Record.
2. The Librarian confirms the check-in processing command.
3. The application engine analyzes timestamp deltas to determine overdue balances and calculate fine rates.
4. The backend system increments available catalog shelf counters and marks the loan record as resolved.

Postcondition: The resource tracking record updates successfully to a closed status flag, updating warehouse inventory levels.

7. User Stories

7.1 Student User Archetype Stories

- *As a Student,* I want to search books across various category titles so that I can quickly verify the physical availability of reading materials before visiting the desk.
- *As a Student,* I want to open my custom profile account metrics screen to track historically compiled borrowing logs and remember previously referenced books.
- *As a Student,* I want to view clear outstanding fee logs so that I can track any accrued fines and resolve balances on time.

7.2 Librarian Archetype Stories

- *As a Librarian,* I want to easily add, modify, or update physical books inside the centralized repository dashboard so that system records accurately reflect reality.
- *As a Librarian,* I want to handle checked-out transactions effortlessly from a unified terminal view so that user checkout wait times are minimized.
- *As a Librarian,* I want to compile and export structured utilization activity reports so that administrative management can evaluate library performance metrics monthly.

7.3 Administrative Management Stories

- *As an Administrator,* I want to manage platform user profiles and enforce role-based access controls so that internal system activities remain secure and auditable.

8. Data Flow Diagram

8.1 Level 0 Context Diagram Definition

The system context model represents a unified data hub processing transactions across external nodes. The system boundaries contain the unified application core, routing structured information read/write signals directly to a secured MySQL Relational Database. This high-level flow pattern operates as follows:

- **Student Nodes:** Direct search queries, profile validation strings, and history parameters inward. They receive indexed catalog collections, account records, and custom fine metrics back.
- **Librarian Nodes:** Route processing instructions, status updates, check-in information, and catalog entries inward. They receive updated transactional confirmations and tracking feedback back.
- **Administrative Nodes:** Inject system configuration profiles and staff account permissions inward. They extract system audit logs and aggregate analytics dashboards out.

8.2 Level 1 Process Model Breakdown

Operational data structures within the system are split into six isolated runtime micro-processes:

1. **Process 1 (Authentication Router):** Parses authorization identities, verifies cryptographic hashed password string keys, and generates system access tokens.
2. **Process 2 (Book Inventory Controller):** Manages CRUD operations for physical text volumes and indexes data schemas into storage.
3. **Process 3 (Member Directory Controller):** Manages data workflows related to member profiles, validation checks, and operational authorization records.
4. **Process 4 (Circulation Transaction System):** Handles the lifecycles of borrowing and check-in workflows, managing changes to availability balances.
5. **Process 5 (Fine Evaluation Processor):** Monitored calculation routine that parses timing variables to compute out-of-bounds fee liabilities.
6. **Process 6 (Reporting Analytics Engine):** Compiles transactional database metrics to generate structured analytical data sheets.

9. Future Enhancements

Planned feature extensions for upcoming version iterations include:

- Integrated QR code asset allocation scanning arrays.
- Hardware scanner tracking optimizations via standard Barcode readers.
- Automated email communication notification delivery pipelines.
- SMS text alert integrations for overdue notifications.
- Cross-platform hybrid mobile client app deployment.
- Online resource booking pre-reservation capabilities.
- AI-powered intelligent recommendation algorithms matching user preferences.

10. Conclusion

The Library Management System delivers a comprehensive software solution designed to eliminate inefficient, paper-heavy tracking workflows through modern automation. By organizing resource catalogs, member profiles, circulation tracking, and fine management into a unified web application, the system minimizes administrative overhead. Built on a robust React, Node.js, and MySQL stack, this platform provides software developers, database architects, and system administrators with a secure, modular, and highly scalable blueprint that aligns with modern software engineering best practices.